Dojo | 36 progressively demanding exercises

This workbook contains 36 original exercises inspired by the techniques in Practice A and by topics named in the syllabus. It is not Practice B and does not predict its questions. Six stages build from traversal to combined stateful problems. Difficulty is cumulative, but individual tasks may feel easier or harder depending on your strengths.

Working rules

Use the detailed guide and A4 sheet for an initial attempt. Keep the separate hints and solutions closed. Before coding, state the input representation, the result, the helper invariant and the progress measure. After coding, test boundaries and the mutation/identity contract, then explain the time and space costs.

Unless overridden, linked inputs are finite, proper and acyclic; numeric operations use the usual unit-cost model; pure predicates/functions do not mutate data or depend on call order. The runtime names follow pair/head/tail/None. Python lists and linked lists are different representations. No built-in sorting/search shortcuts are allowed where the algorithm is the learning target. Each problem's additional restrictions override these defaults.

All reference code uses the documented Source Academy Python §3 grammar and pair/stream primitives. Helpers that are not predeclared are supplied explicitly. Run the copied programs in the official Source Academy playground with Python §3 selected. The included checks are feedback, not an exhaustive assessment autograder.

Six stages

Stage	Exercises	Main skill
1	D01-D06	Traversal and state
2	D07-D12	Transformations, matrices and stacks
3	D13-D18	Destructive updates and identity
4	D19-D24	Search, restoration and changed state
5	D25-D30	Laziness, memoization and self-reference
6	D31-D36	Syllabus transfer and synthesis

Browser practice

Open index.html or the published website. Choose a problem and edit its starter in the browser. Copy its runnable program, open Source Academy, select Python §3, paste and run. The copied program includes the support definitions and checks. Friends need only a web browser. Download any draft you want to keep beyond the current tab session.

Staged help

Hint 1 names the central idea. Hint 2 suggests a structural step. The full solution includes code, rationale and costs. After reading an answer, close it and reimplement the function later; do not count a copied solution as an independent attempt.

1 | Foundations

D01 Count without indexing

Function: count_nodes | **Guide:** G05, G08

Write count_nodes(xs) for a finite linked list. Return its number of nodes. Do not mutate it, convert it to a indexed list, or use a predeclared length function. Use a loop with constant auxiliary space. Before coding, state what your counter means after each iteration.

```
count_nodes(None) == 0
count_nodes(llist(8, 8, -2)) == 3
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D02 Alternating checksum

Function: alternating_total | **Guide:** G06

Return x0 - x1 + x2 - x3 + ... for the values in xs. The empty result is 0. Use one traversal, no mutation, no indexing and no conversion. Values can be negative, so the sign depends on position, not on the value.

```
alternating_total(llist(5, 2, -4, 3)) == -4
alternating_total(None) == 0
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

1 | Foundations

D03 Adjacent gaps, without a first element

Function: adjacent_gaps | **Guide:** G06, G08

Return a fresh linked list of consecutive differences: x1-x0, x2-x1, and so on. Unlike delta encoding, do not copy x0 into the output. For zero or one input value, return None. Do not mutate the input. Do not convert to indexed lists.

```
adjacent_gaps(llist(4, 9, 7, 7)) -> llist(5, -2, 0)
adjacent_gaps(llist(8)) -> None
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D04 Prefix totals with an initial balance

Function: running_totals | **Guide:** G06

Write running_totals(xs, start). Each output is start plus the sum of input values up to that position. Do not include start itself as a separate element. Return a fresh linked list and leave xs unchanged. Empty input returns None. Do not repeatedly recompute prefixes.

```
running_totals(llist(3, -5, 4), 10) -> llist(13, 8, 12)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

1 | Foundations

D05 Most expensive leg

Function: route_peak | **Guide:** G06, G09

Given a nonnegative directed distance matrix and a linked route, return the largest cost of one consecutive leg, not the sum. For an empty or singleton route return 0. House numbers are valid indices; repeated houses are permitted and diagonal entries are 0. Read only; no conversion of route.

```
mat = [[0, 7, 2], [4, 0, 6], [9, 1, 0]]
route_peak(mat, llist(2, 0, 1)) == 9
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D06 Select by position

Function: keep_positions | **Guide:** G07, G08

Write keep_positions(xs, pred), where pred is a pure function of a zero-based position. Return a fresh linked list containing values at positions where pred(i) is true, in their original order. The predicate depends on the index, not the value. No mutation or conversion.

```
keep_positions(llist(8, 9, 10, 11), lambda i: i % 2 == 0)
# -> llist(8, 10)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

2 | Transformations

D07 Compress equal runs

Function: encode_runs | **Guide:** G08, G17

Compress a finite linked list of numbers into a linked list of records. Each record is pair(value, count), NOT llist(value, count). Counts are positive. A run is a maximal consecutive block of equal values. Create fresh outer nodes and records; leave the input untouched. No indexed-list conversion.

```
encode_runs(llist(4, 4, 9, 4))
# -> llist(pair(4, 2), pair(9, 1), pair(4, 1))
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D08 Expand run records

Function: decode_runs | **Guide:** G08

Invert D07. Input is a linked list of pair(value, positive_count) records. Return a fresh linked list with each value repeated count times. Do not mutate records or their outer list. Do not use indexed-list conversion. The output length may greatly exceed the number of records.

```
decode_runs(llist(pair(5, 2), pair(-1, 3)))
# -> llist(5, 5, -1, -1, -1)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

2 | Transformations

D09 Non-overlapping chunk totals

Function: chunk_sums | **Guide:** G06, G08

For positive k, group xs into consecutive non-overlapping chunks of at most k values. Return one sum per chunk as a fresh linked list. Include the final short chunk. Empty input gives None. Do not modify xs or convert it. This groups values; it is not a sliding window.

```
chunk_sums(llist(1, 2, 3, 4, 5), 2) -> llist(3, 7, 5)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D10 Transpose a rectangle

Function: transpose | **Guide:** G04, G09

Given a nonempty rectangular indexed list of nonempty numeric rows, return its transpose. Allocate a fresh outer list and independent output rows. Do not mutate input rows. Your code must work for 1-by-C, R-by-1 and nonsquare matrices. Do not use zip or a library transpose.

```
transpose([[1, 2, 3], [4, 5, 6]])
# -> [[1, 4], [2, 5], [3, 6]]
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

2 | Transformations

D11 Snake across rows

Function: snake_rows | **Guide:** G09

Return a rows-by-cols matrix containing 1 through rows*cols. Fill row 0 from left to right, row 1 from right to left, and continue alternating. Both dimensions are positive. Construct independent rows. Derive the formula instead of reversing the same row object.

```
snake_rows(3, 4)
# -> [[1,2,3,4], [8,7,6,5], [9,10,11,12]]
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D12 Postfix with a unary operator

Function: eval_postfix_extended | **Guide:** G10

Evaluate a valid indexed list of numeric operands, binary + - * / tokens and unary token "neg". neg replaces the top value with its negative. Binary operators consume two values. No division by zero occurs. Use a stack; do not use eval. Do not mutate the token list. / means Python §3 division. Use a preallocated indexed list and a top counter; .append() and .pop() are not part of the §3 grammar.

```
eval_postfix_extended([8, 3, "-", "neg", 2, "/"]) == -2.5
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

3 | Mutation and identity

D13 Reverse the links

Function: d_reverse | **Guide:** G19

Reverse a finite acyclic linked list by changing only existing tails. Return the new first node. No new pairs, no indexed-list conversion, no set_head, and constant auxiliary space. Empty input returns None. Save information before overwriting it. Values and node identities must be preserved.

```
xs = llist(2, 5, 7)
# d_reverse(xs) -> llist(7, 5, 2) using the same three nodes
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D14 Destructive filtering

Function: d_keep | **Guide:** G19

Keep only nodes whose numerical head satisfies a pure pred. Preserve their relative order and exact node identities. Change only tails; allocate no pairs and do not convert the input. The final kept node must point to None. Discarded nodes need not be detached or otherwise preserved. Return None if none qualify.

```
d_keep(llist(5, 2, 8, 3), lambda x: x % 2 == 0)
# -> llist(2, 8), reusing those nodes
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

3 | Mutation and identity

D15 Cut at a precise boundary

Function: d_split_at | **Guide:** G19

Given 0 <= k <= length(xs), split the list into its first k nodes and the remaining nodes. Return a indexed list [first, second] holding the two heads. This two-item result container is allowed; no new linked pairs, copied nodes or head changes are allowed. The first chain must end at None; handle k=0 and k=n.

```
d_split_at(llist(6, 7, 8, 9), 2)
# -> [llist(6, 7), llist(8, 9)] using original nodes
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D16 Merge descending chains

Function: d_merge_desc | **Guide:** G20

Merge two disjoint, acyclic linked lists already sorted in nonincreasing order. Return a nonincreasing chain using every original node once. No pair/llist, head modification or indexed-list conversion. On ties, choose the node from xs first. Empty inputs are valid.

```
d_merge_desc(llist(9, 4, 4), llist(8, 4, 1))
# -> llist(9, 8, 4, 4, 4, 1)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

3 | Mutation and identity

D17 Destructive merge sort

Function: d_merge_sort | **Guide:** G20, G21

Sort an arbitrary finite acyclic linked list in ascending order using merge sort. Reuse all nodes; do not change heads or allocate pairs. You may define a destructive ascending merge helper. The split must physically disconnect the halves. Duplicates must remain stable. No conversion, sorted or .sort.

```
d_merge_sort(llist(4, 1, 4, 2, 0)) -> llist(0, 1, 2, 4, 4)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D18 Stable destructive partition

Function: d_partition | **Guide:** G19, G20

Split xs into nodes passing pure pred and nodes failing it, preserving original order within each chain. Return [yes_head, no_head] as a indexed list. Every original node must appear exactly once across the two disjoint results. Allocate no pairs; modify only tails. Use constant auxiliary space.

```
d_partition(llist(4, 1, 6, 3), lambda x: x >= 4)
# -> [llist(4, 6), llist(1, 3)]
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

4 | Search and state

D19 Can the robot reach the goal?

Function: reachable | **Guide:** G11-G13

On a nonempty rectangular grid of '.', '#' and one 'G', determine whether a path exists from valid open (r,c) to G. Movement is orthogonal. Use recursive exploration and temporary marking. Restore the exact grid before returning, even when a branch succeeds early. Small grids are intended.

```
reachable([[".", "#", "G"], [".", ".", "."]], 0, 0) is True
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D20 Count simple paths

Function: count_simple_paths | **Guide:** G11-G13

Count all paths from an open start to G that never repeat a cell. A path ends when it first reaches G. Two paths differ if their ordered cell sequences differ. Return 0 when unreachable and 1 when already on G. Use temporary marking and restore input. Small grids only.

```
count_simple_paths([[".", "."], [".", "G"]], 0, 0) == 2
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

4 | Search and state

D21 A key changes the state

Function: shortest_with_key | **Guide:** G13

Cells are '.', '#', 'K', 'D' and one 'G'. The robot starts without a key. Entering K permanently grants a reusable key; D can be entered only with it. Return the minimum number of orthogonal moves to G or math_inf. Input must be unchanged. Allow revisiting a coordinate after obtaining the key. Use small grids and an explicit (row,column,key) state.

```
shortest_with_key([[".", "D", "G"], ["K", "#", "#"]], 0, 0) == 4
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D22 Pay on entry

Function: cheapest_path | **Guide:** G12-G13

costs is a rectangular matrix containing positive numbers or None for walls. Starting and goal coordinates are valid traversable cells. Return the minimum total entry cost to reach (gr,gc) from (r,c). Exclude the starting cell's cost and include the goal's cost; start=goal returns 0. Orthogonal moves only. Restore any temporary changes. Return math_inf if unreachable. Small grids are intended.

```
cheapest_path([[1, 9, 1], [1, 1, 1]], 0, 0, 0, 2) == 4
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

4 | Search and state

D23 Break one wall

Function: shortest_one_break | **Guide:** G13

On a '.', '#', 'G' grid, return the fewest moves to G if at most one wall may be entered. A wall consumes that allowance. Start is '.' or 'G'. Orthogonal movement; return math_inf if impossible. Do not mutate the maze. Model state as coordinate plus whether the allowance is used; small grids only.

```
shortest_one_break([[".", "#", "G"]], 0, 0) == 2
shortest_one_break([[".", "#", "#", "G"]], 0, 0) == math_inf
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D24 Exactly k moves, revisits allowed

Function: walks_exactly | **Guide:** G13, G16, G21

Count walks of exactly k orthogonal moves that end on G. k is nonnegative. Repeated cells are allowed, and G may be visited and left before the final move. Walls cannot be entered. Use memoization keyed by row, column and moves remaining; do not mutate the grid. This changed rule deliberately makes the earlier path-marking template inappropriate.

```
walks_exactly([[".", "G"]], 0, 0, 3) == 1
walks_exactly([[".", "G"]], 0, 0, 2) == 0
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

5 | Laziness and sharing

D25 Lazy strided characters

Function: strided_chars | **Guide:** G14-G15

Return a stream of text[start], text[start+step], ... while indices stay in range. start>=0 and step>0. Construction must take O(1) time under the course indexing model, regardless of string length. No slicing or eager conversion. The empty result is None. Traversal forces later nodes.

```
eval_stream(strided_chars("abcdefg", 1, 2), 3)
# -> llist("b", "d", "f")
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D26 Lazy adjacent differences

Function: stream_gaps | **Guide:** G14-G16

Return a stream of consecutive differences of finite or infinite numeric s. Empty and singleton inputs produce None. To decide whether a first difference exists, you may force one source tail at construction. Thereafter use only the required one-element lookahead per output. Memoize output tails; leave source pairs unchanged. Assume source tail forcing is O(1).

```
eval_stream(stream_gaps(stream(2, 8, 5)), 2)
# -> llist(6, -3)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

5 | Laziness and sharing

D27 Lazy running balance

Function: stream_scan | **Guide:** G06, G15-G16

Return cumulative sums of numeric stream s, starting with an external balance total. Do not emit the initial balance separately. Finite inputs end at None; infinite inputs stay productive. Construct at most one output node now and memoize every output tail. Assume one source-tail step costs O(1).

```
eval_stream(stream_scan(stream(3, -1, 5), 10), 3)
# -> llist(13, 12, 17)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D28 Fair interleaving

Function: interleave | **Guide:** G15

Return a stream alternating elements from s then t. When either stream ends, continue with the other. Either may be infinite. Construct in O(1); memoize tails of newly built nodes. Returning a remaining input stream unchanged is allowed. Do not force the source tail until it is needed.

```
interleave(stream(1,2,3), stream(8,9))
# first values: 1, 8, 2, 9, 3
# stream(...) is the predeclared finite-stream constructor
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

5 | Laziness and sharing

D29 Expand a stream of runs

Function: expand_run_stream | **Guide:** G15-G17

Input is a finite or infinite stream of records pair(value, positive_count). Produce a stream repeating each record's value count times. Do not expand whole runs eagerly. Initial construction and each new output step must be O(1), assuming O(1) input-tail forcing. Memoize output tails, especially across run boundaries.

```
expand_run_stream(stream(pair("a",2), pair("b",1)))
# values: "a", "a", "b"
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D30 Self-referential smooth numbers

Function: smooth_numbers | **Guide:** G16-G18

Return an increasing infinite stream beginning at 1 of all positive integers whose prime factors are only 2, 3 and 5, with duplicates removed. Use the relationship S = 1 followed by the ordered union of 2*S, 3*S and 5*S. Define lazy scale and lazy merge helpers; memoize every tail. No precomputed prefix, sorted collection, or eager infinite recursion. The first 1000 values must be practical.

```
# First 12: 1,2,3,4,5,6,8,9,10,12,15,16
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

6 | Transfer and synthesis

D31 Find a boundary, not just a match

Function: lower_bound | **Guide:** G21

For a sorted nondecreasing indexed list a, return the first index i with a[i]>=target, or len(a) if none exists. Empty lists and duplicates are valid. Require O(log(n+1)) time, no slicing, no mutation and no library search. State a half-open search invariant before writing code.

```
lower_bound([1, 3, 3, 7], 3) == 1
lower_bound([1, 3, 3, 7], 8) == 4
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D32 Preserve a nested shape

Function: tree_map | **Guide:** G22

A tree is None, a number, or a proper linked list of trees. Write tree_map(tree,f) applying pure f to every numeric leaf. Preserve all nesting and empty sublists, allocating fresh pairs throughout. Do not flatten, mutate or convert it. is_pair is predeclared in Python §3.

```
tree_map(llist(2, llist(3, None), 4), lambda x: x*x)
# -> llist(4, llist(9, None), 16)
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

6 | Transfer and synthesis

D33 Pass the answer onward

Function: sum_squares_cps | **Guide:** G07, G22

Write sum_squares_cps(xs,k) returning k(sum of squared elements). Follow continuation-passing style: the empty case calls k(0); the recursive call receives a continuation representing the pending work. Do not mutate xs. k need not return a number. Distinguish constant control-stack depth from the memory occupied by pending continuation closures.

```
sum_squares_cps(llist(2,3), lambda v: v+1) == 14
sum_squares_cps(None, lambda v: [v]) == [0]
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D34 A closure with undo

Function: make_undo_counter | **Guide:** G04, G07, G16

make_undo_counter(start) returns a function counter(command,amount). "add" adds amount and records the new value; "undo" removes the most recent add if one exists; "get" reads the value without recording anything. Each command returns the current value. Pass 0 as amount for "get" and "undo"; default parameters are not used. Commands are valid. Separate counters must be independent. Do not use globals.

```
c = make_undo_counter(10)
c("add", 4) == 14
c("add", -2) == 12
c("undo", 0) == 14
c("get", 0) == 14
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

6 | Transfer and synthesis

D35 Evaluate lexical let expressions

Function: eval_let | **Guide:** G23

Evaluate tagged indexed-list syntax: ["num",n], ["var",name], ["add",e1,e2], ["mul",e1,e2], ["let",name,rhs,body]. env is a linked list of pair(name,value) bindings, newest first, initially None, passed explicitly. A let evaluates rhs in the OLD environment, then body in an extended environment. All variables are bound. Do not use eval or mutate syntax/environment. Nested bindings must shadow outer ones only within their body.

```
eval_let(["let", "x", ["num", 2],
    ["let", "x", ["add", ["var", "x"], ["num", 3]],
     ["var", "x"]]], None)
# -> 5
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.

D36 Decode a route and stream its cost

Function: route_cost_stream | **Guide:** G06, G09, G15-G16

mat is a directed nonnegative cost matrix, start a valid house index, and deltas a finite or infinite numeric stream. Each delta shifts the current house index; all resulting houses are valid and may repeat. Emit cumulative travel cost AFTER each move, excluding any initial zero. Build the first result in O(1), advance lazily, memoize output tails, and leave all inputs unchanged. Assume O(1) source steps. Do not first materialize the decoded route.

```
mat = [[0,4,9], [2,0,3], [7,1,0]]
route_cost_stream(mat, 0, stream(1,1,-2))
# values: 4, 7, 14
```

Before coding: state the invariant and one edge case that would defeat a tempting wrong solution.

After coding: check the value, permitted mutations, termination and cost.